

TD 6 – Langage PL/pgSQL

Table des matières

1) Problématique	1
2) Prérequis	1
3) Exercices	2
4) Conclusion.....	10

1) Problématique

Nous souhaitons approfondir nos connaissances en SQL avec le langage procédural de PostgreSQL : le PL/pgSQL. Nous allons donc répondre aux différents exercices du TD 6 en travaillant sur la base de données d'une banque nommée « hhbdirect » et en nous aidant du dossier 3.3 qui nous est fourni.

2) Prérequis

Nous allons devoir utiliser PostgreSQL pour effectuer les différents exercices du TD 6. Pour cela, on ouvre le terminal « psql » en appuyant sur la touche Windows puis en tapant notre mot de passe habituel, pour moi ça sera donc ?init01.

Pour travailler sur ce TD, on se connecte directement à la base de données fournie avec la commande \c hhbdirect. On peut ensuite vérifier à l'aide de la commande \dt que l'ensemble des tables (affecter, agence, client, compte, date, operation, posseder, remunerer, type_compte) sont bien créées et disponibles dans la base :

```
postgres=# \c hhbdirect
Vous êtes maintenant connecté à la base de données « hhbdirect » en tant qu'utilisateur « postgres ».
hhbdirect=# \dt
      Liste des tables
Schéma |  Nom  | Type | Propriétaire
-----+-----+-----+-----
public | affecter | table | postgres
public | agence  | table | postgres
public | client  | table | postgres
public | compte  | table | postgres
public | date    | table | postgres
public | operation | table | postgres
public | posseder | table | postgres
public | remunerer | table | postgres
public | type_compte | table | postgres
(9 lignes)
```

Le terminal est désormais prêt à l'emploi et nous pouvons commencer à effectuer les exercices.

3) Exercices

Exercice 1 : Fonctions `getOneTuple`, `getOneTupleTable` et `lesClients`

Dans cet exercice, on nous donne trois fonctions à tester sur notre table client pour voir comment elles fonctionnent et ce qu'elles retournent.

1. Fonction `getOneTuple()`

On s'aide du code fourni, on déclare une variable `res` de type `record`. On fait un `SELECT INTO res * FROM client` pour mettre le premier tuple de la table client dedans, puis on le retourne :

```
hhbdirect=# CREATE OR REPLACE FUNCTION getOneTuple() RETURNS record
hhbdirect=# LANGUAGE plpgsql
hhbdirect=# AS $$
hhbdirect$$ declare
hhbdirect$$     res record ;
hhbdirect$$ begin
hhbdirect$$     select into res * from client ;
hhbdirect$$     return res ;
hhbdirect$$ end;
hhbdirect$$ $$ ;
CREATE FUNCTION
```

Résultat :

```
hhbdirect=# SELECT getOneTuple();
              getonetuple
-----
(1,DUPONT,Pierre,"5 Rue du Port, 22300 LANNION",Pdupont,Pdupont)
(1 ligne)
```

On voit que le résultat nous renvoie une seule ligne avec l'ensemble des informations du premier client entre parenthèses sous la forme d'un record.

2. Fonction `getOneTupleTable()`

Pour la deuxième fonction, le type de retour est `SETOF client`, ce qui veut dire un ensemble de lignes de la table client. Dans le bloc d'instructions, on utilise `return query select * from client` :

```
hhbdirect=# CREATE OR REPLACE FUNCTION getOneTupleTable()
hhbdirect=# RETURNS SETOF client
hhbdirect=# LANGUAGE plpgsql
hhbdirect=# AS $$
hhbdirect$$ begin
hhbdirect$$     return query select * from client ;
hhbdirect$$ end;
hhbdirect$$ $$ ;
CREATE FUNCTION
```

Résultat :

```
hhbdirect=# SELECT * FROM getOneTupleTable();
```

num_client	nom_client	prenom_client	adresse_client	identifiant_internet	mdp_internet
1	DUPONT	Pierre	5 Rue du Port, 22300 LANNION	Pdupont	Pdupont
2	DUPONT	Annie	5 Rue du Port, 22300 LANNION	Adupont	Adupont
3	DELAVAL	Jean	12 Bd de l'Orne, 14234 OUISTREHAM	Jdelaval	Jdelaval
4	HANOT	Eric	13 Avenue de Neuilly, 94230 NOGENT SUR MARNE	Ehanot	Ehanot
5	LEVY	Sarah	1 Rue Neuve, 14110 CONDE SUR NOIREAU	Slevy	Slevy

(5 lignes)

Ici le résultat est différent car ça nous retourne tous les tuples de la table client sous forme d'un tableau complet avec toutes les colonnes.

3. Fonction lesClients()

Pour la troisième fonction, elle renvoie également un SETOF client mais cette fois-ci avec une boucle FOR res IN select * from client LOOP. On déclare res de type client%ROWTYPE et à chaque tour de boucle on fait un return next res pour ajouter le client au résultat :

```
hhbdirect=# CREATE OR REPLACE FUNCTION lesClients() RETURNS SETOF client
hhbdirect=# LANGUAGE plpgsql
hhbdirect=# AS $$
hhbdirect$$ declare
hhbdirect$$   res client%ROWTYPE ;
hhbdirect$$ begin
hhbdirect$$   for res in select * from client
hhbdirect$$   loop
hhbdirect$$     return next res ;
hhbdirect$$   end loop;
hhbdirect$$   return;
hhbdirect$$ end ;
hhbdirect$$ $$ ;
CREATE FUNCTION
```

Résultat :

```
hhbdirect=# SELECT * FROM lesClients();
```

num_client	nom_client	prenom_client	adresse_client	identifiant_internet	mdp_internet
1	DUPONT	Pierre	5 Rue du Port, 22300 LANNION	Pdupont	Pdupont
2	DUPONT	Annie	5 Rue du Port, 22300 LANNION	Adupont	Adupont
3	DELAVAL	Jean	12 Bd de l'Orne, 14234 OUISTREHAM	Jdelaval	Jdelaval
4	HANOT	Eric	13 Avenue de Neuilly, 94230 NOGENT SUR MARNE	Ehanot	Ehanot
5	LEVY	Sarah	1 Rue Neuve, 14110 CONDE SUR NOIREAU	Slevy	Slevy

(5 lignes)

Le résultat obtenu est identique à la fonction précédente, on obtient bien la liste complète des clients de la table.

Exercice 2 : Nombre de clients habitant dans une ville

On souhaite créer une fonction qui retourne le nombre de clients habitant dans une ville donnée. On met donc la variable nom_ville de type varchar en paramètre de la fonction, et la fonction retourne un entier.

On déclare la variable nb integer. On effectue un SELECT count(*) INTO nb depuis la table client avec la condition WHERE adresse_client LIKE '%'

nom_ville || '%' pour chercher le nom de la ville dans les adresses des clients, puis on le retourne avec le RETURN nb :

```
hhbdirect=# CREATE OR REPLACE FUNCTION nb_clients_ville(nom_ville varchar)
hhbdirect=# RETURNS integer AS $$
hhbdirect$$ DECLARE
hhbdirect$$     nb integer;
hhbdirect$$ BEGIN
hhbdirect$$     SELECT count(*) INTO nb
hhbdirect$$     FROM client
hhbdirect$$     WHERE adresse_client LIKE '%' || nom_ville || '%';
hhbdirect$$
hhbdirect$$     RETURN nb;
hhbdirect$$ END;
hhbdirect$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
hhbdirect=# SELECT nb_clients_ville('LANNION');
 nb_clients_ville
-----
                2
(1 ligne)
```

On teste avec la ville de « LANNION », et la fonction nous renvoie bien 2 car il y a 2 clients qui habitent à Lannion.

Exercice 3 : Nombre de clients débiteurs

Maintenant on va créer une fonction qui retourne le nombre de clients débiteurs, c'est-à-dire les clients qui ont au moins un compte avec un solde négatif (inférieur à 0).

On déclare la fonction nb_clients_debiteurs sans paramètre retournant un entier. On déclare la variable nb integer. On joint la table posseder et la table compte sur le type et le numéro de compte, puis on vérifie que le solde est strictement inférieur à 0. On utilise count(DISTINCT num_client) pour ne pas compter deux fois le même client s'il a plusieurs comptes débiteurs, puis on retourne nb :

```
hhbdirect=# CREATE OR REPLACE FUNCTION nb_clients_debiteurs()
hhbdirect=# RETURNS integer AS $$
hhbdirect$$ DECLARE
hhbdirect$$     nb integer;
hhbdirect$$ BEGIN
hhbdirect$$     SELECT count(DISTINCT num_client) INTO nb
hhbdirect$$     FROM posseder
hhbdirect$$     JOIN compte ON posseder.id_type = compte.id_type AND posseder.num_compte = compte.num_compte
hhbdirect$$     WHERE solde < 0;
hhbdirect$$
hhbdirect$$     RETURN nb;
hhbdirect$$ END;
hhbdirect$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
hhbdirect=# SELECT nb_clients_debiteurs();
nb_clients_debiteurs
-----
1
(1 ligne)
```

Le résultat nous donne 1 car il n'y a qu'un seul client dans la banque dont le solde est négatif.

Exercice 4 : Insérer un tuple dans la table client

Désormais nous allons créer une fonction qui permet d'insérer un tuple dans la table client. On crée donc la fonction `inserer_client` qui va prendre nom, prenom, adresse, identifiant et mdp en paramètres.

On calcule automatiquement le numéro du nouveau client en ajoutant 1 au maximum actuel avec `SELECT max(num_client) + 1 INTO nouveau_num FROM client`. Ensuite on insère le tuple dans la table client. Si l'insertion réussit ça nous retourne true (t), et en cas d'erreur le bloc d'exception retourne false (f) :

```
hhbdirect=# CREATE OR REPLACE FUNCTION inserer_client(
hhbdirect(#      nom varchar,
hhbdirect(#      prenom varchar,
hhbdirect(#      adresse varchar,
hhbdirect(#      identifiant varchar,
hhbdirect(#      mdp varchar
hhbdirect(# )
hhbdirect=# RETURNS boolean AS $$
hhbdirect$$ DECLARE
hhbdirect$$      nouveau_num integer;
hhbdirect$$ BEGIN
hhbdirect$$      SELECT max(num_client) + 1 INTO nouveau_num FROM client;
hhbdirect$$      INSERT INTO client VALUES (nouveau_num, nom, prenom, adresse, identifiant, mdp);
hhbdirect$$      RETURN true;
hhbdirect$$ EXCEPTION
hhbdirect$$      WHEN OTHERS THEN
hhbdirect$$          RETURN false;
hhbdirect$$ END;
hhbdirect$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
hhbdirect=# SELECT inserer_client('MOREAU', 'Julie', '24 Avenue des Fleurs, 14000 CAEN', 'jmoreau', 'azerty456');
inserer_client
-----
t
(1 ligne)

hhbdirect=# SELECT * FROM client WHERE nom_client = 'MOREAU';
 num_client | nom_client | prenom_client |      adresse_client      | identifiant_internet | mdp_internet
-----+-----+-----+-----+-----+-----
7 | MOREAU    | Julie        | 24 Avenue des Fleurs, 14000 CAEN | jmoreau             | azerty456
(1 ligne)
```

On teste l'insertion d'un nouveau client « MOREAU Julie ». La fonction nous retourne bien true (t) puis on vérifie avec un SELECT que la cliente est bien présente dans la table avec le numéro 7.

Exercice 5 : Nombre d'opérations pour un compte et un mois donné

Pour cet exercice, on doit créer la fonction nb_operation_compte_mois() qui permet de calculer le nombre d'opérations effectuées sur un compte pendant un mois et une année donnés.

La fonction prend 4 paramètres entiers : mois, annee, num_c (numéro du compte) et type_c (type de compte). Elle retourne un entier.

Dans la fonction, on déclare une variable nb integer. On fait un SELECT count(*) INTO nb depuis la table operation. On utilise la fonction EXTRACT() pour filtrer le mois et l'année de la colonne date. De plus, comme précisé dans la consigne, un compte peut être soit le compte d'origine, soit la destination d'un versement. On met donc une condition OR pour vérifier les deux cas : (num_compte = num_c AND id_type = type_c) OR (num_compte_vers = num_c AND id_type_vers = type_c). Puis on retourne nb :

```
hhbdirect=# CREATE OR REPLACE FUNCTION nb_operation_compte_mois(
hhbdirect(#      mois integer,
hhbdirect(#      annee integer,
hhbdirect(#      num_c integer,
hhbdirect(#      type_c integer
hhbdirect(# )
hhbdirect-# RETURNS integer AS $$
hhbdirect$$ DECLARE
hhbdirect$$      nb integer;
hhbdirect$$ BEGIN
hhbdirect$$      SELECT count(*) INTO nb
hhbdirect$$      FROM operation
hhbdirect$$      WHERE EXTRACT(MONTH FROM date) = mois
hhbdirect$$      AND EXTRACT(YEAR FROM date) = annee
hhbdirect$$      AND (
hhbdirect$$          (num_compte = num_c AND id_type = type_c)
hhbdirect$$          OR (num_compte_vers = num_c AND id_type_vers = type_c)
hhbdirect$$      );
hhbdirect$$      RETURN nb;
hhbdirect$$ END;
hhbdirect$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
hhbdirect=# SELECT nb_operation_compte_mois(11, 2012, 3, 1);
nb_operation_compte_mois
-----
3
(1 ligne)

hhbdirect=# SELECT nb_operation_compte_mois(10, 2012, 5, 2);
nb_operation_compte_mois
-----
1
(1 ligne)
```

On effectue deux tests : d'abord pour le compte 3 de type 1 en novembre 2012, ce qui donne 3 opérations. Ensuite pour le compte 5 de type 2 en octobre 2012 qui est destinataire d'un virement, et la fonction nous retourne bien 1 opération.

Exercice 6 : Création de l'identifiant et du mot de passe internet

Ici on doit écrire la fonction `creer_id_internet()` qui reçoit en argument le nom et le prénom d'un client et qui va créer son identifiant internet (le mot de passe étant identique à l'identifiant).

Par défaut, l'identifiant est la 1ère lettre du prénom en majuscule avec le nom en minuscule (ex : Pdupont). Mais si un ou des clients portent déjà le même nom et prénom, on doit rajouter à la fin le nombre de clients portant déjà ce nom et prénom (ex : Alambert3).

On déclare la fonction avec `nom` et `prenom` en `varchar`, retournant un `varchar`. On déclare `id_internet` `varchar` et `nb` `integer`. On compte d'abord les clients existants portant ce nom et prénom avec un `count(*)` dans `nb`. Ensuite avec un `IF THEN ELSE` : si `nb = 0`, on concatène la 1ère lettre du prénom en majuscule grâce à `upper(substr(prenom, 1, 1))` avec le nom en minuscule `lower(nom)`. Sinon on concatène la même chose en ajoutant `nb` à la fin. Puis on retourne `id_internet` :

```

hhbdirect=# CREATE OR REPLACE FUNCTION creer_id_internet(nom varchar, prenom varchar)
hhbdirect=# RETURNS varchar AS $$
hhbdirect$# DECLARE
hhbdirect$#     id_internet varchar;
hhbdirect$#     nb integer;
hhbdirect$# BEGIN
hhbdirect$#     SELECT count(*) INTO nb
hhbdirect$#     FROM client
hhbdirect$#     WHERE upper(nom_client) = upper(nom)
hhbdirect$#           AND upper(prenom_client) = upper(prenom);
hhbdirect$#
hhbdirect$#     IF nb = 0 THEN
hhbdirect$#         id_internet := upper(substr(prenom, 1, 1)) || lower(nom);
hhbdirect$#     ELSE
hhbdirect$#         id_internet := upper(substr(prenom, 1, 1)) || lower(nom) || nb;
hhbdirect$#     END IF;
hhbdirect$#
hhbdirect$#     RETURN id_internet;
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION

```

Résultat :

```

hhbdirect=# SELECT creer_id_internet('BERNARD', 'Thomas');
   creer_id_internet
-----
Tbernard
(1 ligne)

hhbdirect=# SELECT creer_id_internet('DUPONT', 'Pierre');
   creer_id_internet
-----
Pdupont1
(1 ligne)

```

On teste avec un nouveau client « Thomas Bernard », ça nous génère bien Tbernard car il n'existe pas encore. Puis on teste avec « Pierre Dupont » qui existe déjà 1 fois dans la table, et ça nous génère bien Pdupont1 !

Exercice 7 : Insertion de toutes les dates d'un mois

Dans cet exercice, on doit écrire la fonction `creer_date()` qui prend en entrée un mois et une année sous forme numérique et qui génère toutes les dates de ce mois pour les insérer dans la table `date`.

La fonction prend deux arguments entiers `mois` et `annee`, et retourne `void`. On déclare une variable `date_courante` de type `date`.

On initialise `date_courante` avec le premier jour du mois grâce à `to_date(annee || '-' || mois || '-01', 'YYYY-MM-DD')`. Ensuite on utilise une boucle `WHILE` : tant que le mois de `date_courante` (obtenu avec `extract(month from date_courante)`) reste égal au mois demandé, on insère la date avec `INSERT INTO date VALUES (date_courante)`, puis on ajoute un jour en faisant `date_courante := date_courante + 1`. Dès que la date passe au mois suivant, la boucle se termine automatiquement :

```

hhbdirect=# CREATE OR REPLACE FUNCTION creer_date(mois integer, annee integer)
hhbdirect=# RETURNS void AS $$
hhbdirect$# DECLARE
hhbdirect$#     date_courante date;
hhbdirect$# BEGIN
hhbdirect$#     date_courante := to_date(annee || '-' || mois || '-01', 'YYYY-MM-DD');
hhbdirect$#
hhbdirect$#     WHILE extract(month from date_courante) = mois LOOP
hhbdirect$#         INSERT INTO date VALUES (date_courante);
hhbdirect$#         date_courante := date_courante + 1;
hhbdirect$#     END LOOP;
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION

```

Résultat :

```

hhbdirect=# SELECT creer_date(2, 2024);
 creer_date
-----
(1 ligne)

hhbdirect=# SELECT count(*), min(date), max(date) FROM date WHERE extract(year from date) = 2024 AND extract(month from date) = 2;
count | min      | max
-----+-----+-----
    29 | 2024-02-01 | 2024-02-29
(1 ligne)

hhbdirect=# SELECT * FROM date WHERE extract(year from date) = 2024 AND extract(month from date) = 2 LIMIT 4;
 date
-----
2024-02-01
2024-02-02
2024-02-03
2024-02-04
(4 lignes)

```

On teste avec le mois de février 2024 (année bissextile). On vérifie ensuite dans la table date avec un SELECT : les 29 jours du mois du 01/02/2024 au 29/02/2024 ont bien été générés et insérés.

Exercice 8 : Création d'un rôle PostgreSQL pour chaque client

Pour ce dernier exercice, on doit écrire une fonction creer_user_client() qui a pour but de créer un compte utilisateur PostgreSQL (un ROLE) pour chaque client de la banque.

L'identifiant de connexion et le mot de passe doivent être l'id_internet et le mot de passe internet du client. La fonction doit vérifier dans la table système pg_user si l'utilisateur n'existe pas déjà avant de le créer, et elle doit retourner le nombre d'utilisateurs créés.

On déclare une variable cl de type record, nb integer initialisé à 0, et existe integer. On utilise une boucle FOR cl IN SELECT identifiant_internet, mdp_internet FROM client WHERE identifiant_internet IS NOT NULL LOOP pour parcourir chaque client. Pour chaque client, on vérifie avec SELECT count(*) INTO existe FROM pg_user WHERE username = lower(cl.identifiant_internet). Si existe = 0, on exécute dynamiquement la commande CREATE ROLE avec EXECUTE, puis on incrémente nb. A la fin on retourne nb :

```

hhbdirect=# CREATE OR REPLACE FUNCTION creer_user_client()
hhbdirect=# RETURNS integer AS $$
hhbdirect$# DECLARE
hhbdirect$#     cl record;
hhbdirect$#     nb integer := 0;
hhbdirect$#     existe integer;
hhbdirect$# BEGIN
hhbdirect$#     FOR cl IN SELECT identifiant_internet, mdp_internet FROM client WHERE identifiant_internet IS NOT NULL LOOP
hhbdirect$#         SELECT count(*) INTO existe FROM pg_user WHERE username = lower(cl.identifiant_internet);
hhbdirect$#         IF existe = 0 THEN
hhbdirect$#             EXECUTE 'CREATE ROLE ' || quote_ident(cl.identifiant_internet) || ' WITH LOGIN PASSWORD ''' || cl.mdp_internet || '''';
hhbdirect$#             nb := nb + 1;
hhbdirect$#         END IF;
hhbdirect$#     END LOOP;
hhbdirect$#     RETURN nb;
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION

```

Résultat :

```

hhbdirect=# SELECT creer_user_client();
  creer_user_client
-----
                6
(1 ligne)

hhbdirect=# \du
                                Liste des rôles
Nom du rôle | Attributs
-----+-----
Adupont      |
Ehanot       |
Jdelaval     |
Pdupont      |
Slevy        |
lmartin      |
postgres    | Superutilisateur, Créer un rôle, Créer une base, Réplication, Contournement RLS

hhbdirect=# SELECT creer_user_client();
  creer_user_client
-----
                0
(1 ligne)

```

Lors de la première exécution, la fonction crée bien les 6 rôles et retourne 6. On vérifie avec la commande \du que les utilisateurs ont bien été créés. Puis on relance la fonction une deuxième fois : elle retourne 0 car tous les rôles existent déjà dans la base.

4) Conclusion

Pour conclure, ce TP m'a permis d'approfondir mes connaissances sur le langage procédural PL/pgSQL avec PostgreSQL. J'ai pu apprendre à manipuler des fonctions plus avancées avec des curseurs, des requêtes dynamiques avec EXECUTE, des structures de boucles conditionnelles WHILE et FOR, ainsi que l'administration avec la création automatisée de comptes utilisateurs. Ces notions me seront très utiles pour la suite de ma formation en BTS SIO !